Programming in Java

ASSIGNMENT 5 – CO5

DATA PERSISTENCE ENGINEERING – FILES AND DATABASES

Course Outcome – CO5: Java I/O – byte, character and buffered streams, the File class, serialization; JDBC – architecture and drivers, Statement, PreparedStatement, CallableStatement, transactions, batch processing and metadata (Sessions 39–48)
Nature: Individual
Duration: One Week
Total Marks: 50

Bloom's Levels Assessed: K4 – Analyse and K5 – Evaluate (the level of each task is stated under the task heading)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA09-<RegisterNumber>-Assignment5** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the Java source code, input files and output screenshots.
- Include a README.md with your name, register number, Java version, the steps to compile and run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Assessment Components

PART A – Persistence Design & Pseudocode: 25 Marks
PART B – Implementation & Experimental Investigation: 25 Marks

PURPOSE OF THE ASSIGNMENT

Almost every real application must store data so that it survives after the program ends. Java offers several ways to do this – text files, binary streams, object serialization and relational databases through JDBC.

This assignment investigates a fundamental question:

> **How should a Java engineer decide where and how data is stored, and how can the program be made correct, safe and efficient as the data grows?**

You are expected to demonstrate the following progression:

> **Model → Choose a Store → Design → Protect → Implement → Measure → Compare → Decide**

Simply obtaining a program that saves and loads data is not sufficient. Your design reasoning, experimental evidence and interpretation are all part of the assessment.

PART A – PERSISTENCE DESIGN & PSEUDOCODE — 25 Marks

All pseudocode must follow the SIMATS Engineering – Standard for Algorithmic Pseudocode.

TASK A1 – DESIGN A FILE FORMAT — 7 Marks

Bloom's Level: K4 – Analyse

Transport Office Bus-Pass Records

The university transport office issues bus passes. Each pass record contains: pass ID, student register number, name, route number, boarding stop, validity date and amount paid. About 8,000 passes are issued each year. The office wants the records stored in a file and needs to (i) add a new pass, (ii) find a pass by pass ID, and (iii) list all passes on a given route.

A1.1 – Represent the Record | 2 Marks

Design the record layout for (a) a delimited text file and (b) a binary file written with DataOutputStream. State the field order, delimiter or field sizes, and how a name containing the delimiter is handled.

A1.2 – Design the Operations | 3 Marks

Write pseudocode for the three operations for the text-file design, stating which stream classes (FileReader/FileWriter, BufferedReader/BufferedWriter, FileInputStream/FileOutputStream) you would use and why.

A1.3 – Design for Damaged Data | 2 Marks

Explain how your design detects and handles a damaged line, a missing file and an incomplete write caused by a power failure.

Alternative Concept and Complexity Analysis

If the passes were stored in a RandomAccessFile with fixed-length records, so that record i is found by seeking to i × recordSize, instead of a delimited text file, what would the solution be? Analyse the time to find a pass by ID (O(n) scan versus O(1) seek) and the space wasted by padding fixed-length fields.

Evidence Expected

Two record layouts, pseudocode for three operations with justified stream choices, and a design for damaged data.

TASK A2 – DON'T READ WHAT YOU DON'T NEED 5 Marks

Bloom's Level: K4 – Analyse

A program reads a 50 MB attendance log one byte at a time using FileInputStream.read() to count the number of lines.

A2.1 – Predict the Cost | 2 Marks

Predict, with reasons, how many read calls the program makes and why it is slow.

A2.2 – Redesign Using Buffering | 2 Marks

Redesign the program using a buffered stream or a byte array buffer, and explain how buffering changes the number of calls to the operating system.

A2.3 – When Buffering Does Not Help | 1 Mark

Describe one situation in which adding a buffer gives little or no benefit.

Alternative Concept and Complexity Analysis

If the whole file were read at once using Files.readAllLines() instead of a buffered stream, what would the solution be? Analyse the time complexity and the memory used (O(file size) versus O(buffer size)) for a 50 MB file, and state when each approach is appropriate.

Evidence Expected

A reasoned prediction, a redesigned read strategy and a justified limitation.

TASK A3 – DISCOVER THE DATA-ACCESS DESIGN PATTERN 5 Marks

Bloom's Level: K4 – Analyse

A student project has three classes – StudentDB, CourseDB and MarksDB. Each one opens a connection, builds an SQL string, executes it, reads the ResultSet into objects and closes the connection. Large parts of the three classes are almost identical.

A3.1 – Identify the Common Pattern | 2 Marks

Identify the steps that are repeated in all three classes and the parts that differ.

A3.2 – Design a Generic Data-Access Interface | 2 Marks

Design a generic interface such as Dao<T, K> with methods for insert, findById, findAll, update and delete, and show how StudentDao implements it.

A3.3 – Map Objects to Tables | 1 Mark

Show how the fields of the Student class map to the columns of the student table, including the primary key.

Alternative Concept and Complexity Analysis

If the SQL were written directly inside the service or GUI classes instead of a generic Dao<T, K>, what would the solution be? Compare the number of places that must change when a table

column is renamed, and analyse whether the time and space cost per database operation differs between the two designs.

Evidence Expected

An identified common pattern, a generic data-access design and an object–table mapping.

TASK A4 – WHAT SHOULD BE SERIALIZED? 4 Marks

Bloom's Level: K4 – Analyse

A hostel management program saves its in-memory state by serializing a Hostel object, which holds a list of Room objects, each holding a list of Resident objects. Each Resident has a name, register number, password and a reference to a live database Connection.

A4.1 – Decide What to Persist | 2 Marks

Decide which fields must be marked transient and why, and explain what happens to the object graph when the Hostel object is serialized.

A4.2 – Plan for Change | 2 Marks

A new field (phone number) is added to Resident after files have already been saved. Explain what happens when an old file is read, and how serialVersionUID affects the outcome.

Alternative Concept and Complexity Analysis

If the hostel data were saved as a CSV or JSON text file instead of Java serialization, what would the solution be? Compare the file size, the time to save and load n residents, and how each approach copes when a new field is added.

Evidence Expected

A justified transient design and a correct explanation of object-graph serialization and versioning.

TASK A5 – DESIGN FOR CORRECTNESS AND SAFETY 4 Marks

Bloom's Level: K4 – Analyse

A5.1 – Transaction Design | 2 Marks

A student transfers ₹500 from a hostel wallet to a canteen wallet. Write pseudocode using JDBC commit and rollback so that money is never lost or created if the program fails between the two updates.

A5.2 – Injection Analysis | 2 Marks

The login check is written as:

```
String sql = "SELECT * FROM users WHERE id='" + id + "' AND pwd='" + pwd + "'";
```

Explain how a crafted input can make the query succeed without a valid password, and redesign the check using PreparedStatement.

Alternative Concept and Complexity Analysis

If the two wallet updates were executed as two separate auto-commit statements instead of one transaction, what would the outcome be when the program fails between them? Analyse the time cost of a transaction versus two independent updates, and the risk to data correctness.

Evidence Expected

Correct transaction pseudocode and a reasoned injection analysis with a safe redesign.

PART B – IMPLEMENTATION & EXPERIMENTAL INVESTIGATION 25 Marks

Part B assesses whether your design reasoning survives contact with actual computation. You are expected to:

Implement → Instrument → Stress → Measure → Compare → Interpret

Execution screenshots alone are insufficient.

TASK B1 – IMPLEMENT AND INSTRUMENT THE FILE STORE 7 Marks

Bloom's Level: K4 – Analyse

Implement the bus-pass file store designed in Task A1 (text format) and the buffered line counter from Task A2.

B1.1 – Validate the Implementation | 2 Marks

Test add, find and list using at least one normal case, one pass ID that does not exist and one damaged line. Explain what each test verifies.

B1.2 – Compare with Your Design | 1 Mark

Compare your implementation with your Part A pseudocode and explain any change you had to make.

B1.3 – Measure the Effect of Buffering | 2 Marks

Measure the time to count lines in files of 1 MB, 10 MB and 50 MB with the unbuffered and buffered versions. Compare the results with your prediction in A2.1.

B1.4 – Measure Growth | 2 Marks

Measure the time to find a pass as the file grows from 1,000 to 100,000 records, and explain the trend.

Complete the Code

Complete TODO 1 below and execute the program.

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
import java.util.Arrays;

public class PassStore {
    private final Path file;

    PassStore(Path file) { this.file = file; }

    public void add(String passId, String regNo, String name, int route, double amount)
            throws IOException {
        try (BufferedWriter w = Files.newBufferedWriter(file,
                StandardOpenOption.CREATE, StandardOpenOption.APPEND)) {
            w.write(String.join(",", passId, regNo, name,
                    String.valueOf(route), String.valueOf(amount)));
            w.newLine();
        }
    }

    /** Returns the five fields of the pass with the given ID, or null if not found. */
    public String[] find(String passId) throws IOException {
        // TODO 1: Read the file line by line with a BufferedReader inside try-with-
        //   resources; skip damaged lines (not exactly 5 fields); return the fields of
        //   the matching pass

        return null;
    }

    public static void main(String[] args) throws IOException {
        Path f = Files.createTempFile("passes", ".csv");
        PassStore store = new PassStore(f);
        store.add("P1001", "192311001", "Arun", 12, 4500);
        Files.writeString(f, "P1002,DAMAGED LINE\n", StandardOpenOption.APPEND);
        store.add("P1003", "192311003", "Divya", 7, 3800);
        System.out.println(Arrays.toString(store.find("P1003")));
        System.out.println(Arrays.toString(store.find("P1002")));
        Files.delete(f);
    }
}
```

Expected output:

```
[P1003, 192311003, Divya, 7, 3800.0]
null
```

Explain why the damaged line for P1002 does not stop the search.

Alternative Concept and Complexity Analysis

Your find() scans the file every time. If the file were read once at start-up into a HashMap<String, String[]> keyed by pass ID, what would the solution be? Analyse the time for q searches on n passes (O(q × n) versus O(n + q)) and the memory used in both approaches.

Evidence Expected

Working code, test table, measured buffering and growth evidence, and interpretation.

TASK B2 – ENGINEER AN ADVERSARIAL INPUT 5 Marks

Bloom's Level: K4 – Analyse

Using a **local test database created only for this assignment**, implement the unsafe login check from Task A5.2 and the PreparedStatement version.

- Construct an input that makes the unsafe version accept an invalid login, and show that the PreparedStatement version rejects it
- Construct a damaged bus-pass file (wrong number of fields, non-numeric amount, empty line) and show how your file store responds
- Explain what each adversarial input reveals about the original design

Alternative Concept and Complexity Analysis

If, instead of PreparedStatement, the program escaped quote characters in the input or validated it with a regular expression before building the SQL string, what would the solution be? Explain why this is weaker, and analyse the time cost of validation per query compared with a PreparedStatement.

Evidence Expected

Deliberately constructed adversarial inputs, observed behaviour of the unsafe and safe versions, and a reasoned explanation.

TASK B3 – STATEMENT vs PREPAREDSTATEMENT vs BATCH 8 Marks

Bloom's Level: K4 – Analyse

Move the bus-pass records into a MySQL table using JDBC.

B3.1 – Correctness on Small Data | 2 Marks

Insert 10 records using each approach and verify, using a SELECT and ResultSetMetaData, that the table contents and column types are correct.

B3.2 – Increase the Data Size | 2 Marks

Insert 1,000, 10,000 and 100,000 records using (a) Statement, (b) PreparedStatement and (c) PreparedStatement with addBatch()/executeBatch(). Record the time for each.

B3.3 – Auto-commit vs Transaction | 2 Marks

Repeat the batch insert with auto-commit on and with a single transaction. Then force a failure half-way and show the table contents in both cases.

B3.4 – Engineering Trade-Off | 2 Marks

Explain the differences you observed, and state which approach you would use for (a) a nightly import of 100,000 passes and (b) a single pass issued at the counter.

Complete the Code

Complete TODO 1 to TODO 3 below, create the bus_pass table in your test database, and call loadAll() with 1,000, 10,000 and 100,000 generated passes.

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.List;

public class PassLoader {
    /** Loads all passes in one transaction, in batches of 1,000 rows.
     *  Returns the number of rows loaded. */
    public static int loadAll(Connection con, List<String[]> passes)
            throws SQLException {
        String sql = "INSERT INTO bus_pass(pass_id, reg_no, name, route_no, amount) "
                   + "VALUES (?, ?, ?, ?, ?)";
        boolean oldAutoCommit = con.getAutoCommit();
        con.setAutoCommit(false);
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            int n = 0;
            for (String[] p : passes) {
                // TODO 1: Set the five parameters from p (route as int, amount as
                //   double), add the row to the batch, and execute the batch after every
                //   1,000 rows


            }
            ps.executeBatch();
            // TODO 2: Make all inserted rows permanent and return the number of rows
            //   loaded

        } catch (SQLException | RuntimeException e) {
            // TODO 3: Undo every insert made in this transaction, then re-throw the
            //   exception

        } finally {
            con.setAutoCommit(oldAutoCommit);
        }
    }
}
```

Insert a pass with an invalid amount half-way through the list and show that no rows remain in the table after the failure.

Alternative Concept and Complexity Analysis

If, instead of PreparedStatement batches, the rows were inserted using one multi-row INSERT statement or the MySQL LOAD DATA command, what would the solution be? Analyse the total time for n rows and the memory used for a batch of b rows in each approach, using your measurements.

Evidence Expected

Verified correctness, measured comparison at three sizes, transaction evidence and a reasoned trade-off.

TASK B4 – FINAL ENGINEERING DECISION 5 Marks

Bloom's Level: K5 – Evaluate

The transport office will now serve all 12 campuses of the university, with 100× the current number of passes and several counters issuing passes at the same time.

B4.1 – Implement a Practical Strategy | 2 Marks

Implement a migration program that reads the existing pass file and loads it into the database using batches and a transaction, reporting the records that failed validation.

B4.2 – Inspect the Database | 1 Mark

Use DatabaseMetaData to report the database product, driver version and the tables present, and explain why this is useful when the application is moved to another server.

B4.3 – Production Decision | 2 Marks

Decide whether the office should use text files, serialization or a database for the new scale. Justify your decision using your measurements and the requirements for concurrent counters, searching and safety.

Alternative Concept and Complexity Analysis

If each campus kept its own serialized file instead of sharing one database, what would the solution be for the 12-campus requirement? Analyse the time to search all passes and the space used, and explain how concurrent counters would be handled in each design.

Evidence Expected

A working migration with a failure report, metadata output and an evidence-based production decision.

MARK DISTRIBUTION

Component	Task	Bloom's Level	Marks
Part A – Design	A1 – File Format Design	K4 – Analyse	7
	A2 – Buffered I/O Design	K4 – Analyse	5
	A3 – Data-Access Pattern	K4 – Analyse	5
	A4 – Serialization Design	K4 – Analyse	4
	A5 – Transactions & Injection	K4 – Analyse	4
	Part A Total		**25**
Part B – Implementation	B1 – Instrumented File Store	K4 – Analyse	7
	B2 – Adversarial Inputs	K4 – Analyse	5
	B3 – Statement vs PreparedStatement vs Batch	K4 – Analyse	8
	B4 – Final Engineering Decision	K5 – Evaluate	5
	Part B Total		**25**
	TOTAL		**50**

SUBMISSION REQUIREMENTS

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, with input files, output screenshots and a README.md.

For **Part A**, submit the record layouts, pseudocode, diagrams and reasoning. Java code will not be accepted in place of pseudocode.

For **Part B**, submit Java source code, the SQL script used to create the test database, test inputs, expected behaviour, actual outputs, measurement data and interpretation. A screenshot showing only a final message is not sufficient evidence.

Students are expected to use the same engineering workflow throughout the assignment:

> **UNDERSTAND → DESIGN → PROTECT → IMPLEMENT → MEASURE → CHALLENGE → INTERPRET**

The key question is not simply “Does the program save the data?” It is also:

How safely, how efficiently and at what scale does it save the data, and what should an engineer change when the s